Лабораторна робота №2. Система типів CTS, менеджмент пам’яті та обробка даних за допомогою методів розширення LINQ

Мета: Дослідити архітектуру пам'яті (Stack/Heap), опанувати механізми передачі функціональності через делегати та лямбда-вирази, а також навчитися використовувати методи розширення LINQ для маніпуляції колекціями.

Індивідуальне завдання

1. Використайте напрацювання за індивідуальним проєктом з лабораторної роботи 1.
2. Дослідити роботу структур: оголосіть у проекті Core структуру (struct) для невеликої сутності (наприклад, Point або Price). Створіть метод, що приймає цю структуру, змінює її поля всередині, і доведіть, що поза методом об'єкт залишився незмінним.
3. Виконати аналіз Boxing/Unboxing: створіть змінну типу object і присвойте їй int. Використовуючи клас System.Diagnostics.Stopwatch, порівняйте швидкість додавання 1,000,000 елементів у нетипізований ArrayList та узагальнений List<int>. Виведіть різницю у мілісекундах.
4. Створити колекцію об'єктів: ініціалізуйте список List<T> (де T — ваш клас з індивідуальної теми) мінімум 10 різними об'єктами. Використовуйте конструктори, створені раніше.
5. Застосувати фільтрацію LINQ (Where): використовуючи метод розширення .Where() та лямбда-вираз, виберіть об'єкти за певним критерієм (наприклад, всі замовлення, сума яких перевищує 1000).
6. Застосувати сортування LINQ (OrderBy): відсортуйте список за двома властивостями одночасно, використовуючи ланцюжок методів .OrderBy(x => ...).ThenBy(x => ...).
7. Здійснити проекцію даних (Select): створіть новий список, що містить лише одну властивість ваших об'єктів (наприклад, лише імена клієнтів або назви товарів) за допомогою методу .Select().

8. Виконати пошук елемента: знайдіть у списку перший об'єкт, що відповідає унікальній умові, за допомогою методу .FirstOrDefault(). Обробіть ситуацію, якщо об'єкт не знайдено (перевірка на null).

9. Вивести результати: оформіть результати всіх LINQ-запитів у вигляді таблиць у консолі, використовуючи цикли foreach.

Теоретичні відомості

Методи розширення (Extension Methods): Це механізм, що дозволяє «додавати» нові методи до вже існуючих типів (наприклад, до List або string) без зміни їхнього вихідного коду. Саме як набір методів розширення для інтерфейсу IEnumerable<T> і реалізована технологія LINQ.

Делегати та Лямбда-вирази: LINQ-запити потребують критеріїв для фільтрації чи сортування. Ці критерії передаються як аргументи-функції:

1. **Делегати:** типи, що вказують на методи.

2. **Анонімні методи:** методи без імені, визначені прямо в коді.

3. **Лямбда-вирази (x => x.Id > 0):** найсучасніший і стислий спосіб запису анонімних методів. Знак => читається як «переходить у».

Менеджмент пам'яті:

- **Stack:** зберігає значущі типи (struct, int). Копіюється значення.
- **Heap:** зберігає об'єкти класів. Копіюється лише адреса (посилання).

Процес пакування (Boxing/Unboxing) передбачає перетворення (перенесення) значущого типу в об'єкт для збереження в купі та зворотне розпакування.

Контрольні питання

1. Чому LINQ називають набором методів розширення? До якого інтерфейсу вони «додаються»?

2. Поясніть структуру лямбда-виразу (n => n.Price > 0). Що тут є вхідним параметром, а що тілом методу?

3. Чим відрізняється делегат від звичайного методу?

4. Чому передача великої структури (struct) у метод може бути менш ефективною, ніж передача класу?

5. В який момент відбувається очищення об'єктів у Heap (купі)?

6. Що станеться, якщо викликати метод .First(), а елемента, що відповідає умові, у списку не існує?